

1º Prompt Gemini

Un protocolo de revisión de código enfocado en **seguridad contra inyecciones SQL (SQLi)** consiste en verificar sistemáticamente cómo se construyen y ejecutan las consultas a la base de datos desde el backend.

1. Protocolo de Revisión de Código (Paso a Paso)

1. Identificar la capa de acceso a datos:

2.

- Localizar todos los archivos o módulos encargados de realizar consultas SQL (repositorios, controladores, servicios o modelos).

○

3. Filtrar por construcción de consultas:

4.

- Buscar cadenas de texto que utilicen concatenación (+, ., `format()`, f-strings, plantillas de texto) para armar una sentencia SQL.

○

5. Validar el uso de parámetros:

6.

- Asegurar que cualquier dato proporcionado por el usuario (en la solicitud HTTP, cabeceras, parámetros de URL o cuerpo del mensaje) entre a la consulta únicamente a través de marcadores de posición (*placeholders*) o parámetros predeterminados de la librería o ORM.

○

7. Revisar consultas dinámicas o compuestas:

8.

- En búsquedas avanzadas o filtros dinámicos, comprobar que los nombres de tablas o columnas no provengan directamente del usuario. Si es necesario, deben validarse contra una lista blanca (*allowlist*).

○

9. Auditar procedimientos almacenados (*Stored Procedures*):

10.

- Verificar que los procedimientos almacenados utilicen parámetros de entrada y no ejecuten código SQL dinámico internamente (`EXEC` o `sp_executesql` concatenando variables).

○

2. Aspectos Clave a Revisar

- **Concatenación directa de texto:** Cualquier consulta que combine texto estático con variables mediante concatenación manual es un punto crítico de riesgo.
-
- **Manejo de caracteres especiales:** Confiar únicamente en funciones de escape de caracteres o filtros manuales suele ser insuficiente o propenso a errores.
-
- **Lógicas ORM inseguras:** Aunque los ORM reducen el riesgo de SQLi, el uso de consultas nativas (*raw queries*) dentro del ORM o el paso de objetos de filtro no desinfectados pueden reintroducir la vulnerabilidad.
-
- **Privilegios de conexión:** Comprobar que el usuario de la base de datos configurado en el backend tenga únicamente los permisos mínimos necesarios para operar (Principio de Menor Privilegio).
-

3. Cómo Evitar Inyecciones SQL

Sentencias Preparadas (Consultas Parametrizadas)

Es la defensa principal. Separa el código de la consulta de los datos de entrada, impidiendo que la entrada del usuario modifique la estructura SQL.

- **Ejemplo inseguro (Concatenación):**
-

```
Python
# Vulnerable a SQLi
f"SELECT * FROM usuarios WHERE id = '{user_id}'"
```

-
- **Ejemplo seguro (Parametrizado):**
-

```
Python
# Seguro: el motor de BD trata 'user_id' estrictamente como un dato
"SELECT * FROM usuarios WHERE id = %s"
```

Uso Apropiado de ORM

Los Mapeadores Objeto-Relacional (como Hibernate, Entity Framework, Prisma o SQLAlchemy) utilizan consultas parametrizadas por defecto en la mayoría de sus métodos estándar.

- **Recomendación:** Priorizar métodos nativos del ORM (*find*, *filter*, *where*) sobre consultas SQL personalizadas o nativas dentro del framework.
-

Listas Blancas (*Allowlists*) para Elementos Estructurales

Dado que los marcadores de posición generalmente no se pueden usar para nombres de tablas o columnas, se deben validar estos valores contra una lista fija permitida.

- **Ejemplo seguro para ordenación dinámica:**
-

```
Python
```

```
"nombre" "fecha" "precio"
```

```
# Se asegura de que la columna seleccionada sea válida antes de incluirla
```

```
if
```

```
in
```

```
f"SELECT * FROM productos ORDER BY {columna_orden}"
```

-

Principio de Menor Privilegio

Asegurar que la cuenta con la que el backend se conecta a la base de datos no tenga permisos administrativos (**DBA** / **root**), sino solo permisos de lectura/escritura en las tablas estrictamente necesarias.